

Redis - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

REDIS — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
TOPIC 1: INSTALLATION	5
1. Introduction	5
1.1 What is it, and why?	5
1.2 The exact problem it solves	5
2. FULL STEPS: Installing and Running Redis Locally (Using Docker — the Easiest, Most Consistent Way)	5
STEP 1 — Run Redis via Docker	5
STEP 2 — Verify It's Running	5
STEP 3 — Try Your First Commands	5
3. Alternative: Native Installation (Linux/Mac)	6
4. Key Configuration (redis.conf)	6
5. Real Life Analogy	6
6. Common Mistakes	6
7. Frequently Asked Interview Questions	6
TOPIC 2: DATA STRUCTURES	7
1. Introduction	7
2. Strings — The Simplest Type	7
3. Lists — Ordered Collections (Like a Java LinkedList)	7
4. Sets — Unordered, Unique Collections (Like a Java HashSet)	7
5. Sorted Sets — Unique Members, Each with a SCORE (Ranked Data)	7
6. Hashes — A Map/Object Within a Single Key (Like a Java HashMap Stored as ONE Redis key)	7
7. Data Structure Selection Table	8
8. Real Life Analogy	8
9. Frequently Asked Interview Questions	8
TOPIC 3: CACHING	8
1. Introduction	8

2. The Cache-Aside Pattern (The Standard Approach)	9
3. Cache Invalidation — "The Hardest Problem in Computer Science"	9
4. Common Caching Strategies	9
5. Real Life Analogy	9
6. Edge Cases	10
7. Frequently Asked Interview Questions	10

TOPIC 4: PUB/SUB 10

1. Introduction	10
2. Code Example (via redis-cli, Conceptually)	10
3. Real Use Case — Simple Real-Time Notifications	11
4. Critical Limitation — Pub/Sub Is "Fire and Forget"	11
5. Real Life Analogy	11
6. Frequently Asked Interview Questions	11

TOPIC 5: EXPIRATION 11

1. Introduction	12
2. Code Examples	12
3. Real Use Case — Rate Limiting	12
4. Real Life Analogy	12
5. Edge Cases	12
6. Frequently Asked Interview Questions	12

TOPIC 6: SPRING BOOT INTEGRATION 13

1. Introduction	13
2. FULL STEP-BY-STEP Setup	13
STEP 1 — Add the Dependency	13
STEP 2 — Configure the Connection	13
STEP 3 — Enable Caching	13
STEP 4 — The Simplest Possible Approach: @Cacheable	13
STEP 5 — Direct RedisTemplate Usage (When You Need More Control)	14
3. @Cacheable vs Manual RedisTemplate — When to Use Which	14
4. Real Life Analogy	14
5. Edge Cases	15

6. Common Mistakes	15
7. Frequently Asked Interview Questions	15
8. Revision Notes (Entire Redis Section Consolidated)	16
9. Homework (Covers Entire Redis Section)	16
FINAL SUMMARY — REDIS COMPLETE	16

REDIS — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: INSTALLATION

1. Introduction

1.1 What is it, and why?

Redis ("REmote DIctionary Server") is an **in-memory data store** — unlike PostgreSQL (which persists data primarily to DISK), Redis keeps its ENTIRE dataset in RAM, making reads/writes extraordinarily fast (sub-millisecond), typically used as a **cache**, a **session store**, or a lightweight **message broker** (Pub/Sub) sitting IN FRONT OF a slower primary database.

1.2 The exact problem it solves

A typical database query (even a fast one) involves disk I/O, query planning, and network round-trips — often taking single-digit to double-digit MILLISECONDS. For data that's read VERY frequently but changes RARELY (a product catalog, a user's session, a leaderboard), repeatedly hitting the full database for the SAME data is wasteful. Redis, keeping data in memory, serves these repeated reads in UNDER a millisecond, dramatically reducing load on your primary database and improving response times.

2. FULL STEPS: Installing and Running Redis Locally (Using Docker — the Easiest, Most Consistent Way)

Since you've just learned Docker, this is genuinely the SIMPLEST way to run Redis locally, avoiding OS-specific installation quirks (Redis doesn't officially support native Windows binaries):

STEP 1 — Run Redis via Docker

```
docker run -d --name my-redis -p 6379:6379 redis:7-alpine
```

STEP 2 — Verify It's Running

```
docker ps # confirm "my-redis" shows as running
docker exec -it my-redis redis-cli # open Redis's own command-line client, INSIDE the container
```

STEP 3 — Try Your First Commands

```
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> SET greeting "Hello, Redis!"
OK
127.0.0.1:6379> GET greeting
"Hello, Redis!"
```

3. Alternative: Native Installation (Linux/Mac)

```
# Ubuntu/Debian
sudo apt-get update && sudo apt-get install redis-server
redis-server # starts the Redis server
redis-cli # connects to it, in a separate terminal

# Mac (via Homebrew)
brew install redis
brew services start redis
```

On WINDOWS, Docker (Step 1-3 above) is the officially recommended approach — Redis Labs does not provide official native Windows builds.

4. Key Configuration (`redis.conf`)

```
port 6379 # default port
maxmemory 256mb # cap how much RAM Redis is allowed to use
maxmemory-policy allkeys-lru # what to do when that limit is hit (evict Least Recently Used keys)
requirepass yourStrongPassword # require authentication (CRITICAL for any non-local deployment!)
```

5. Real Life Analogy

- **Restaurant:** If your main database (PostgreSQL) is the restaurant's full, walk-in cold storage (has EVERYTHING, but takes time to walk to and retrieve from), Redis is like a small, INSTANTLY-accessible countertop shelf holding just the few ingredients used in EVERY single dish — grabbing from the countertop is nearly instant, compared to walking to the cold storage room each time.

6. Common Mistakes

- Running Redis WITHOUT `requirepass` (authentication) on anything beyond a purely local, trusted development machine — a genuinely common, serious real-world security vulnerability (unsecured Redis instances have been a repeated target for attackers).
- Not setting a `maxmemory` limit in production, risking Redis consuming ALL available server RAM.

7. Frequently Asked Interview Questions

- 1 **What is Redis?** An in-memory data store, commonly used as a cache, session store, or lightweight message broker, offering sub-millisecond read/write performance.
- 2 **Why is Redis so much faster than a typical disk-based database for repeated reads?** It keeps the entire dataset in RAM, avoiding disk I/O entirely for reads/writes.
- 3 **What's the recommended way to run Redis locally on Windows?** Via Docker (`docker run redis:7-alpine`), since Redis has no official native Windows build.
- 4 **Why is `requirepass` important?** Unsecured, publicly-reachable Redis instances (with no authentication) have been a common, serious real-world attack target — authentication should always be configured outside of purely local, trusted development.

TOPIC 2: DATA STRUCTURES

1. Introduction

What is it? Unlike a simple key-value cache, Redis supports RICH DATA STRUCTURES natively — Strings, Lists, Sets, Sorted Sets, and Hashes — each with its OWN specialized commands, letting you model real application data structures directly in Redis rather than just caching flat strings.

2. Strings — The Simplest Type

```
SET user:1:name "Alice"
GET user:1:name # "Alice"
INCR page:views # atomically increments a NUMERIC string value by 1
EXPIRE user:1:name 3600 # sets a TTL (Topic 5)
```

3. Lists — Ordered Collections (Like a Java `LinkedList`)

```
LPUSH recent_orders "order-101" # push to the LEFT (front)
RPUSH recent_orders "order-102" # push to the RIGHT (back)
LRANGE recent_orders 0 -1 # get ALL elements (0 to -1 = start to end)
LPOP recent_orders # remove and return the FIRST element
```

Real use case: a simple activity feed, or a job QUEUE (workers `RPOP/BLPOP` from one end while producers `LPUSH` to the other).

4. Sets — Unordered, Unique Collections (Like a Java `HashSet`)

```
SADD user:1:tags "premium" "verified"
SMEMBERS user:1:tags # {"premium", "verified"}
SISMEMBER user:1:tags "premium" # 1 (true) - 0(1) membership check
SINTER user:1:tags user:2:tags # set INTERSECTION - tags common to BOTH users
```

Real use case: tracking unique visitors, tag systems, "mutual friends" style intersections.

5. Sorted Sets — Unique Members, Each with a SCORE (Ranked Data)

```
ZADD leaderboard 1500 "alice"
ZADD leaderboard 2200 "bob"
ZADD leaderboard 1800 "carol"
ZREVRANGE leaderboard 0 2 WITHSCORES # TOP 3 players, highest score first
ZRANK leaderboard "alice" # alice's current RANK (position) in the leaderboard
```

Real use case: leaderboards, rate-limiting (scoring by timestamp), priority queues.

6. Hashes — A Map/Object Within a Single Key (Like a Java `HashMap` Stored as ONE Redis key)

```
HSET user:1 name "Alice" email "alice@example.com" age "30"
HGET user:1 name # "Alice"
HGETALL user:1 # the ENTIRE hash as field-value pairs
HINCRBY user:1 loginCount 1 # atomically increments ONE field within the hash
```

Real use case: storing a whole OBJECT (like a user profile) under ONE key, more memory-efficient than separate keys per field.

7. Data Structure Selection Table

Structure	Use When
String	Simple values, counters, cached JSON blobs
List	Ordered data, simple queues, recent-activity feeds
Set	Unique membership checks, tag systems
Sorted Set	Ranked/scored data — leaderboards, priority queues
Hash	An entire object's fields grouped under one key

8. Real Life Analogy

- **Restaurant:** A String is like a single sticky note ("Table 5 = Reserved"). A List is like an ORDER QUEUE ticket rail (first-in-first-out). A Set is like a list of "VIP guest names tonight" (no duplicates, fast "is this person on the list?" checks). A Sorted Set is like a WAITLIST ranked by exact arrival time. A Hash is like ONE customer's complete order card (name, table, dish, special requests) all grouped together under one ticket number.

9. Frequently Asked Interview Questions

- 1 **Name the five core Redis data structures.** String, List, Set, Sorted Set, Hash.
 - 2 **What's a real-world use case for a Sorted Set?** A leaderboard — members are ranked automatically by their associated score.
 - 3 **Why would you use a Hash instead of storing each field as a separate String key?** To group an entire object's fields under ONE key, which is more memory-efficient and lets you update/retrieve fields atomically as a related group.
 - 4 **What's the time complexity of `SISMEMBER` (checking Set membership)?** $O(1)$ — Redis Sets are implemented as hash tables internally.
-
-

TOPIC 3: CACHING

1. Introduction

What is it? Caching is Redis's MOST COMMON use case — storing the RESULT of an expensive operation (a slow database query, a complex calculation) so SUBSEQUENT requests for the SAME data can be served INSTANTLY from memory, instead of repeating that expensive work.

2. The Cache-Aside Pattern (The Standard Approach)

```
1. Application receives a request for data (e.g., "get product #42")
2. CHECK Redis first: does "product:42" exist in the cache?
YES -> return the CACHED value immediately (FAST - a "cache hit")
NO -> query the DATABASE (SLOWER - a "cache miss"),
then STORE the result in Redis for NEXT time,
then return it to the caller
```

```
public Product getProduct(Long id) {
    String cacheKey = "product:" + id;
    String cached = redisTemplate.opsForValue().get(cacheKey);

    if (cached != null) {
        return deserialize(cached); // CACHE HIT - fast path
    }

    Product product = productRepository.findById(id).orElseThrow(); // CACHE MISS - slow path
    redisTemplate.opsForValue().set(cacheKey, serialize(product), Duration.ofMinutes(10)); // populate cache
    return product;
}
```

3. Cache Invalidation — "The Hardest Problem in Computer Science"

When the UNDERLYING data changes (e.g., the product's price is updated), the CACHED copy becomes STALE unless explicitly handled:

```
public Product updateProduct(Long id, Product updated) {
    Product saved = productRepository.save(updated);
    redisTemplate.delete("product:" + id); // INVALIDATE the stale cached entry
    return saved;
}
```

4. Common Caching Strategies

Strategy	How It Works
Cache-Aside (shown above)	Application code explicitly checks/populates the cache
Write-Through	Every WRITE goes to the cache AND the database simultaneously, keeping them always in sync
Write-Behind	Writes go to the cache immediately; the database is updated ASYNCHRONOUSLY later (faster writes, but a real risk of data loss if the cache fails before syncing)

5. Real Life Analogy

- **Restaurant:** Caching is like a chef keeping today's MOST popular pre-made sauce sitting ready on the counter (cache), instead of cooking it fresh from raw ingredients (the database) for EVERY single order — MUCH faster to serve, but if the "official" recipe changes mid-shift (data update), that countertop batch must be THROWN OUT and remade (cache invalidation), or customers will get the OLD, wrong version.

6. Edge Cases

Case	Result
Forgetting to invalidate the cache after an UPDATE	Callers keep receiving STALE, outdated data until the cache entry naturally expires (Topic 4) — a common, serious real-world bug
A "thundering herd" — many requests simultaneously experience a cache miss for the SAME key at once	ALL of them hit the database simultaneously, potentially overwhelming it — mitigated with techniques like request coalescing/locking

7. Frequently Asked Interview Questions

- 1 **What is the cache-aside pattern?** The application checks the cache first; on a miss, it queries the database, then populates the cache for future requests.
- 2 **What is "cache invalidation," and why is it hard?** Removing/updating stale cached data when the underlying source changes — hard because it requires reliably tracking EVERY code path that could modify the data, and forgetting even one causes silently stale reads.
- 3 **What's the difference between write-through and write-behind caching?** Write-through updates the cache AND database synchronously together; write-behind updates the cache immediately and syncs the database asynchronously later (faster, but riskier if the cache fails before syncing).

TOPIC 4: PUB/SUB

1. Introduction

What is it? Redis Pub/Sub (Publish/Subscribe) is a lightweight MESSAGING pattern — publishers send messages to a named CHANNEL, and any number of subscribers listening to that channel receive them IMMEDIATELY, in real time.

2. Code Example (via `redis-cli`, Conceptually)

```
# Terminal 1 (subscriber):  
SUBSCRIBE notifications  
  
# Terminal 2 (publisher):  
PUBLISH notifications "New order placed: #1042"  
  
# Terminal 1 IMMEDIATELY receives:  
1) "message"  
2) "notifications"  
3) "New order placed: #1042"
```

3. Real Use Case — Simple Real-Time Notifications

```
@Service
public class NotificationPublisher {
    private final StringRedisTemplate redisTemplate;

    public void notifyNewOrder(String orderId) {
        redisTemplate.convertAndSend("order-notifications", "New order: " + orderId);
    }
}

@Component
public class NotificationSubscriber implements MessageListener {
    @Override
    public void onMessage(Message message, byte[] pattern) {
        System.out.println("Received: " + message.toString());
        // e.g., push this to connected WebSocket clients for a live notification
    }
}
```

4. Critical Limitation — Pub/Sub Is "Fire and Forget"

Unlike Kafka (covered in a later section), Redis Pub/Sub does NOT persist messages. If a subscriber isn't CONNECTED at the exact moment a message is published, that message is LOST FOREVER for that subscriber — there's no replay, no message history. This makes Redis Pub/Sub suitable for lightweight, real-time, "nice to have" notifications, but NOT for critical events that must never be missed (use a proper message queue like Kafka or RabbitMQ for those).

5. Real Life Analogy

- **Restaurant:** Redis Pub/Sub is like a kitchen's LIVE, spoken announcement ("Order 42 is ready for pickup!") — any staff member CURRENTLY within earshot hears it instantly, but anyone who stepped away for even a moment MISSES it entirely, with no way to "replay" that announcement later.

6. Frequently Asked Interview Questions

- 1 **What is Redis Pub/Sub?** A lightweight publish/subscribe messaging pattern — publishers send messages to named channels, delivered immediately to any currently-connected subscribers.
- 2 **What's the critical limitation of Redis Pub/Sub compared to a proper message queue like Kafka?** Messages are NOT persisted — if a subscriber isn't connected at the moment of publishing, that message is lost forever, with no replay capability.
- 3 **When is Redis Pub/Sub an appropriate choice?** Lightweight, real-time, non-critical notifications where occasionally missing a message is acceptable — not for critical events requiring guaranteed delivery.

TOPIC 5: EXPIRATION

1. Introduction

What is it? Expiration (TTL — Time To Live) lets you set an AUTOMATIC deadline after which Redis DELETES a key entirely — essential for caching (Topic 3), session management, and rate-limiting, ensuring stale/temporary data doesn't accumulate forever.

2. Code Examples

```
SET session:abc123 "user-data-here" EX 1800 # expires in 1800 seconds (30 minutes)
SETEX session:abc123 1800 "user-data-here" # equivalent, older syntax
EXPIRE existing_key 3600 # set/update a TTL on an ALREADY-existing key
TTL session:abc123 # check REMAINING seconds until expiration
PERSIST session:abc123 # REMOVE the expiration - key becomes permanent again
```

3. Real Use Case — Rate Limiting

```
public boolean isRateLimited(String userId) {
    String key = "rate-limit:" + userId;
    Long requestCount = redisTemplate.opsForValue().increment(key);

    if (requestCount == 1) {
        redisTemplate.expire(key, Duration.ofMinutes(1)); // set the window's expiry on the FIRST request
    }

    return requestCount > 100; // more than 100 requests within this 1-minute window -> rate limited
}
```

4. Real Life Analogy

- **Restaurant:** Expiration is like a "best before" timestamp automatically stamped on any freshly-prepared item sitting on the countertop cache — once that time passes, the kitchen AUTOMATICALLY discards it, ensuring nothing sits around indefinitely getting stale, without a human needing to manually check and throw it out.

5. Edge Cases

Case	Result
Setting an EXPIRE on a key that doesn't exist	Has no effect — Redis returns 0 (indicating nothing happened)
Reading TTL on a key with NO expiration set	Returns -1
Reading TTL on a key that doesn't exist at all	Returns -2

6. Frequently Asked Interview Questions

- 1 **What does TTL stand for in Redis?** Time To Live — how long, in seconds, until a key automatically expires and is deleted.
- 2 **What does `TTL somekey` return if the key exists but has no expiration set?** -1.
- 3 **What does `TTL somekey` return if the key doesn't exist at all?** -2.
- 4 **Give a real production use case for key expiration besides caching.** Rate limiting — tracking request counts within a rolling time window that automatically resets.

TOPIC 6: SPRING BOOT INTEGRATION

1. Introduction

What is it? Spring Boot provides `spring-boot-starter-data-redis`, giving you `RedisTemplate` for direct Redis operations, AND Spring's `@Cacheable` annotation (an AOP-based mechanism, exactly like `@Transactional` from the Spring Core AOP topic) for ZERO-BOILERPLATE caching directly on your service methods.

2. FULL STEP-BY-STEP Setup

STEP 1 — Add the Dependency

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

STEP 2 — Configure the Connection

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
spring.cache.type=redis
```

STEP 3 — Enable Caching

```
@SpringBootApplication
@EnableCaching // enables @Cacheable/@CacheEvict AOP processing
public class MyApplication { }
```

STEP 4 — The Simplest Possible Approach: `@Cacheable`

```
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id") // AUTOMATIC caching - Spring handles EVERYTHING!
    public Product getProduct(Long id) {
        System.out.println("Fetching from DATABASE..."); // only prints on a CACHE MISS
        return productRepository.findById(id).orElseThrow();
    }

    @CacheEvict(value = "products", key = "#id") // AUTOMATICALLY invalidates the cache entry
    public Product updateProduct(Long id, Product updated) {
        return productRepository.save(updated);
    }

    @CacheEvict(value = "products", allEntries = true) // clears the ENTIRE cache region
    public void clearAllProductCache() { }
}
```

Notice: NO manual `redisTemplate.get()/set()` calls anywhere — `@Cacheable` (built on Spring AOP, exactly like `@Transactional`) automatically wraps the method: checks the cache first, and only runs the actual method body (hitting the database) on a genuine cache MISS, automatically storing the result afterward.

STEP 5 — Direct `RedisTemplate` Usage (When You Need More Control)

```
@Service
public class SessionService {
    private final RedisTemplate<String, String> redisTemplate;

    public SessionService(RedisTemplate<String, String> redisTemplate) {
        this.redisTemplate = redisTemplate;
    }

    public void createSession(String sessionId, String userId) {
        redisTemplate.opsForValue().set("session:" + sessionId, userId, Duration.ofMinutes(30));
    }

    public String getUserForSession(String sessionId) {
        return redisTemplate.opsForValue().get("session:" + sessionId);
    }

    public void useSortedSet(String leaderboardKey, String player, double score) {
        redisTemplate.opsForZSet().add(leaderboardKey, player, score);
    }
}
```

3. `@Cacheable` vs Manual `RedisTemplate` — When to Use Which

Approach	Best For
<code>@Cacheable/@CacheEvict</code>	Simple "cache this method's result" scenarios — the vast majority of real caching needs
<code>RedisTemplate</code> (direct)	Using Redis's rich data structures directly (Lists, Sets, Sorted Sets), Pub/Sub, rate limiting, or fine-grained control over caching logic

4. Real Life Analogy

- **Restaurant:** `@Cacheable` is like having a fully AUTOMATED countertop system that transparently remembers and instantly re-serves a dish's recent result WITHOUT the chef needing to think about it at all — the chef just cooks normally, and the system handles all the "have I made this recently?" logic invisibly. `RedisTemplate` is like the chef DIRECTLY and DELIBERATELY managing that countertop shelf themselves, for more specialized needs (a live order queue, a real-time announcement system) beyond simple result caching.

5. Edge Cases

Case	Result
@Cacheable on a method whose return value ISN'T serializable	Fails at runtime — Redis needs to serialize the object to store it (JSON via Jackson, or Java serialization, depending on configuration)
Calling a @Cacheable-annotated method from WITHIN the SAME class (this.method())	Does NOT go through caching — the EXACT same AOP proxy limitation covered in Spring Core (Topic 10) and Spring Boot's @Async (Topic 10)

6. Common Mistakes

- Forgetting @EnableCaching — @Cacheable/@CacheEvict annotations are silently ignored without it.
- Not invalidating (@CacheEvict) the cache on updates, leading to stale data being served indefinitely.
- Calling a @Cacheable method internally within the same class, unaware it bypasses caching entirely due to the AOP proxy limitation.

7. Frequently Asked Interview Questions

- 1 **What does @Cacheable do?** Automatically wraps a method with caching logic (via Spring AOP) — checking the cache first, only executing the method body on a genuine cache miss, and storing the result afterward.
- 2 **What must be enabled for @Cacheable to work?** @EnableCaching on a configuration/main application class.
- 3 **What does @CacheEvict do?** Removes (invalidates) a cache entry, typically used alongside update/delete operations to prevent serving stale data.
- 4 **When would you use RedisTemplate directly instead of @Cacheable?** When you need Redis's richer data structures (Sorted Sets, Lists), Pub/Sub, or fine-grained caching control beyond simple method-result caching.
- 5 **Why doesn't @Cacheable work when called from within the same class?** The same Spring AOP proxy limitation as @Transactional/@Async — only externally-invoked calls go through the proxy that implements the caching behavior.

8. Revision Notes (Entire Redis Section Consolidated)

REDIS — CHEAT SHEET

=====

IN-MEMORY data store - sub-millisecond reads/writes, used as cache/session store/lightweight message broker

INSTALL (Windows): `docker run -d -p 6379:6379 redis:7-alpine` (no native Windows build)

DATA STRUCTURES:

String -> SET/GET/INCR simple values, counters

List -> LPUSH/RPUSH/LRANGE/LPOP ordered, queues, feeds

Set -> SADD/SMEMBERS/SISMEMBER unique membership, O(1) checks

Sorted Set -> ZADD/ZREVRANGE/ZRANK ranked/scored data, leaderboards

Hash -> HSET/HGET/HGETALL an object's fields under ONE key

CACHING: cache-aside pattern (check cache -> miss -> query DB -> populate cache) is the standard approach. MUST invalidate on writes or data goes stale.

PUB/SUB: PUBLISH/SUBSCRIBE - real-time but FIRE-AND-FORGET, NOT persisted (unlike Kafka) - a disconnected subscriber MISSES messages entirely

EXPIRATION: EX/EXPIRE/TTL - TTL returns -1 (no expiry set) or -2 (key doesn't exist)

SPRING BOOT: @EnableCaching + @Cacheable(value=, key=) / @CacheEvict - ZERO-boilerplate caching via Spring AOP (same proxy mechanism/limitation as @Transactional/@Async - internal this.method() calls bypass it!) RedisTemplate for direct data-structure/Pub-Sub access

9. Homework (Covers Entire Redis Section)

- 1 Run Redis via Docker, connect with `redis-cli`, and practice at least 2 commands from each of the 5 data structures.
- 2 Implement the cache-aside pattern manually (without `@Cacheable`) for a simple `getProduct(id)` method, printing "Fetching from DATABASE" only on a genuine cache miss.
- 3 Add `@Cacheable/@CacheEvict` to a Spring Boot service method, and verify (via the database-fetch print statement) that a second call for the SAME id skips the database entirely.
- 4 Implement a simple rate limiter using `INCR + EXPIRE`, and test that it correctly blocks a user after exceeding the configured request limit within the time window.

FINAL SUMMARY — REDIS COMPLETE

This concludes the **Redis** section — 6 topics:

- 1 **Installation** — Docker-based local setup (the recommended path, especially on Windows).
- 2 **Data Structures** — String, List, Set, Sorted Set, Hash, with real use cases for each.
- 3 **Caching** — the cache-aside pattern, and why cache invalidation is genuinely hard.
- 4 **Pub/Sub** — real-time messaging, and its critical "fire and forget, not persisted" limitation vs Kafka.
- 5 **Expiration** — TTL mechanics, rate-limiting as a real production use case.
- 6 **Spring Boot Integration** — `@Cacheable/@CacheEvict` (zero-boilerplate, AOP-based) vs direct `RedisTemplate` usage.

Redis is the caching layer that sits in front of everything you've built across the JDBC/PostgreSQL/Hibernate/Spring Data JPA sections — reducing database load and dramatically improving response times for frequently-read data. Next: Kafka, covering the persistent, guaranteed-delivery messaging system that Redis Pub/Sub explicitly is NOT.

(END OF REDIS MASTER NOTES — ready to proceed to Kafka whenever you say "Next Topic.")